Project Guideline CSE332 (TNF)

Here you will be implementing your own CPU based on the "Machine Code Bits" mentioned in the pdf file, submitted by individuals. The CPU must be able to execute the instructions stored in the memory one after another. Implement your datapath and control units properly so that your Datapath can execute instructions automatically. You have to submit the **Complete Datapath based on IAS with your assigned bits and operations.**

Submission must include:

- One Logisim file containing the whole Complete Datapath. **(register file design, ALU design, control unit and ALU control, instruction and data memory)**
- **Project Report:**
    - A document containing the table of Control Unit and ALU control.
    - The final version of your ISA based on the operations assigned to you in the pdf in same order.
    - ***You should mention:***
        1. *How many operands*
        2. *Types of operand?* ***(Register based or Memory based??)***
        3. *How many operations?* ***(*****Note that your ISA must be based on what instructions you were given and in which order they were given.*****)***
        4. *Types of operations?* ***(Arithmetic, logical, branch type?? How many from each category? List the opcodes and respective binary values)***
        5. *No. of the format of instruction* ***(how many different formats?)***
        6. *Describe each of the formats* ***(fields and field length)***

Note: It is difficult and error-prone to write machine code manually. The problem can be addressed by writing an assembler, which can automatically generate a machine code from an assembly file. In this project, you might write an assembler for your ISA. The assembler reads a program written using assembly language in a text file, then translates it into binary code and generates an output file(.txt) containing machine code. The generated output files will be useful to run a program when you develop your actual CPU.

Language: You can use any high-level language. A demo assembler created with Python is attached here for your reference. You are strongly advised to use them to save your time. You might need to modify the existing functions/classes according to your need.



Project Report

23 BIT SINGLE CYCLE MIPS CPU

To,

Tanjila Farah (TnF)

From,

Course: CSE332.4

Group: 9

Mohd Atik Akbar Rahat(2211757042)

Rakib Rayhan(2132332642)

Introduction

Developing a custom Central Processing Unit (CPU) based on the Instruction Set Architecture (ISA) is a fundamental aspect of computer architecture. This project involves the design and implementation of a CPU capable of executing instructions stored in memory sequentially. The CPU's design is inspired by the IAS (Institute for Advanced Study) architecture, incorporating elements such as a datapath,control units, an Arithmetic Logic Unit (ALU), and both instruction and data memory.This project emphasizes a modular approach to CPU design, integrating components such as:

1. **Register File:** Stores intermediate data and results.
2. **ALU:** Performs arithmetic and logical operations.
3. **Control Unit:** Decodes instructions and generates control signals for execution.
4. **Instruction and Data Memory:** Holds the program and associated data.

The primary objective is to create a functional CPU capable of:

1. Fetching instructions from memory.
2. Decoding instructions into meaningful operations.
3. Executing operations based on the ISA-defined formats.
4. Writing results back to memory or registers.

The final deliverables for this project include:

- A **Logisim file** showcasing the complete datapath design with all required components.
- A **Project Report** containing:
    - The control unit table.
    - ALU control table.
    - Detailed ISA documentation with descriptions of formats, operands, operations, and opcodes.

ISA:

An Instruction Set Architecture (ISA) is part of the abstract model of a computer that defines how the CPU is controlled by the software. The ISA acts as an interface between the hardware and the software, specifying both what the processor is capable of doing as well as how it gets done.

The ISA has three types of instructions (R-type, I-type, and J-type), each with a specific operand structure:

R-Type: 3 operands (rs, rt, rd)

I-Type: 2 operands (rs, rt) + immediate value

J-Type: 1 operand (address)

Types of operands:

- **Register-based**: Operands specified by `rs`, `rt`, and `rd` fields in R-type and I-type instructions.
- **Memory-based**: For instructions like `LW` (load word) and `SW` (store word), which involve accessing memory.
- **Immediate-based**: For I-type instructions such as `ADDi` (add immediate).
- **Address-based**: For J-type instructions such as `JMP` (jump).

Number of Operations:

Based on the instructions provided, assume we are working with 16 total operations:

Arithmetic: 4 operations (add, sub, addi,slt).

Logical: 5 operations (and,or,nor,sll,srl).

Branch: 3 operations (beq, bne,jmp).

Memory: 2 operations (lw, sw).

Other: 1 operation (nop).

The ISA supports **15 operations** based on the instructions given in the specified order:

Instruction	Type	Work
LW	I	Store word in memory.
ADDi	I	Set if less than.
NOP		Logical shift left.
NOR	R	Subtract two registers.
BEQ	I	Jump to a specified address.
OR	R	Bitwise AND.
ADD	R	Load word from memory.
BEN	I	Add immediate value to a register.
SRL	R	No operation.
SW	I	Bitwise NOR.
SLT	R	Branch if two registers are equal.
SLL	R	Bitwise OR.
SUB	R	Add two registers.
JMP	J	Branch if two registers are not equal.
ADD	R	Logical shift right.

Bit Breakdown

	Op : 3	**Rs : 5**	**Rt : 5**	**Rd : 5**	**Shamt : 2**	**Funct : 3**
R :	22-20	19-15	14-10	9-5	4-3	2-0
	Op : 3	**Rs : 5**	**Rt : 5**	**Immidt : 10**		
I :	22-20	19-15	14-10	9-0		
	Op : 3	**Address : 20**				
J :	22-20	19-0				

Control Unit Table

Opcode	000	001	010	011	100	101	110	111
Control Signal	**R-Type**	**LW**	**ADDi**	**BEQ**	**BNE**	**SW**	**JMP**	**NOP**
RegDst	1	0	0	X	X	X	X	0
ALUSrc	0	1	1	0	0	1	X	X
MemToReg	0	1	0	X	X	X	X	X
RegWrite	1	1	1	0	0	0	0	0
MemRead	0	1	0	0	0	0	0	0
MemWrite	0	0	0	0	0	1	0	0
Branch	0	0	0	1	1	0	0	0
Jump	0	0	0	0	0	0	1	0
ALUOp	10	00	00	01	01	00	XX	XX

Opcode Table

Instruction	Type	Opcode (3 bits)
All R-type	R-type	000
LW	I-type	001
ADDi	I-type	010
BEQ	I-type	011
BNE	I-type	100
SW	I-type	101
JMP	J-type	110
NOP		111

Function Table

Instruction	funct (3 bits)
ADD	000
SUB	001
AND	010
OR	011
NOR	100
SLT	101
SLL	110
SRL	111

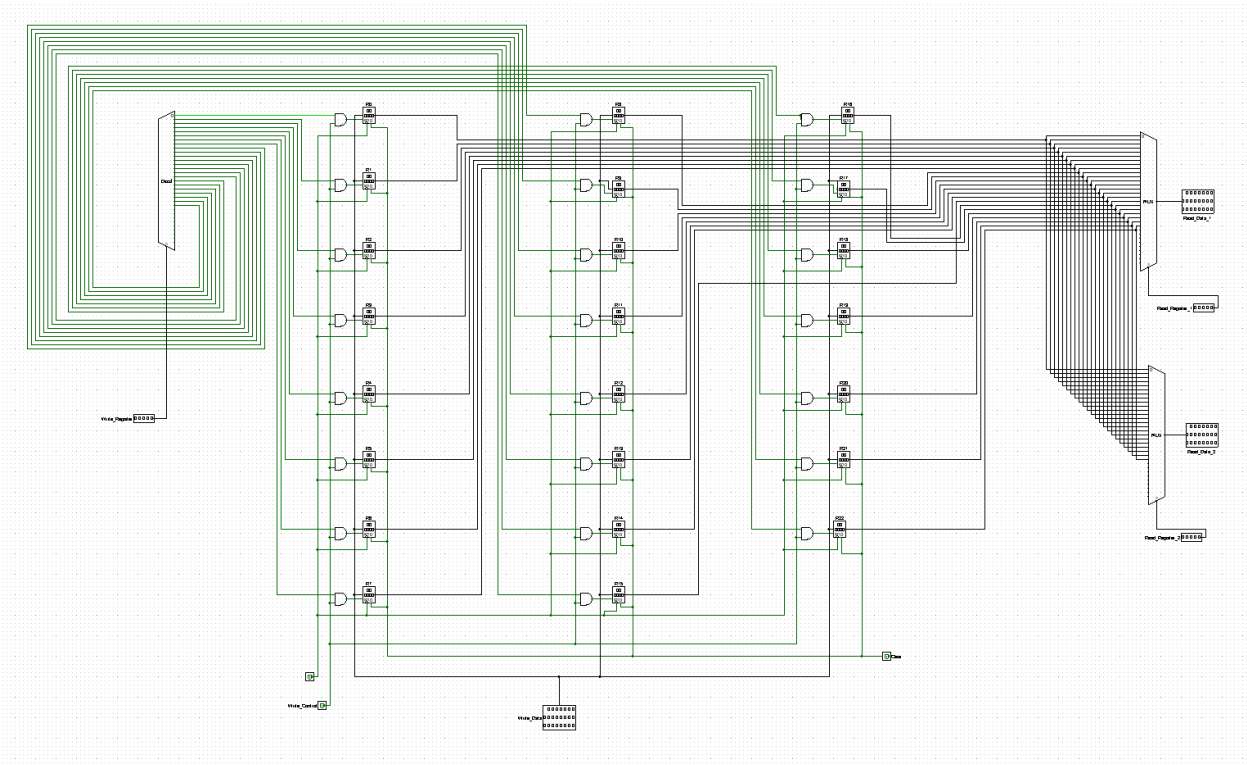
ALU Control Table

ALUOp (2 bits)	funct (3 bits)	ALU Control Output	Operation
00	XXX	000	Add (for LW, SW)
01	XXX	001	Subtract (for BEQ)
10	000	000	Add
10	001	001	Subtract
10	010	010	AND
10	011	011	OR
10	100	100	NOR
10	101	101	Set 1 if Less Than
10	110	110	Shift Left Logical
10	111	111	Shift Right Logical

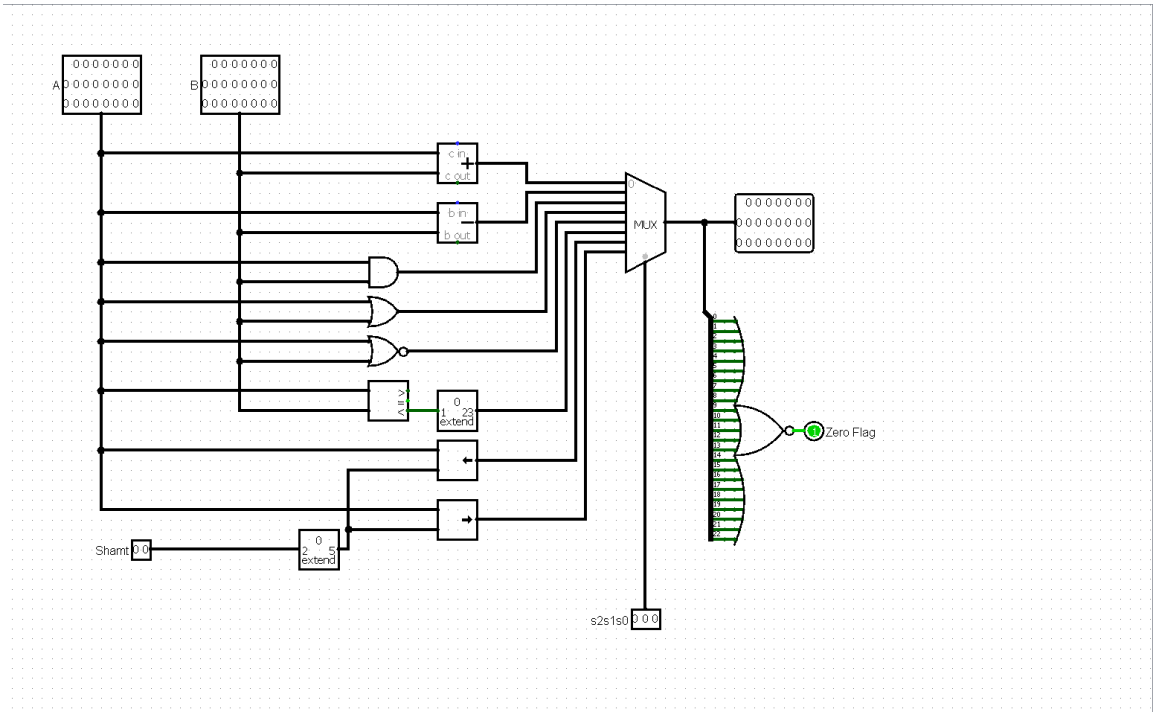
Instruction Set

Instruction	LW	ADDi	NOP	NOR	BEQ	OR	ADD	BNE	SRL	SW	SLT	SLL	SUB	JMP	AND
Opcode	**001**	**010**	111	000	**011**	000	000	**100**	000	**101**	000	000	000	**110**	000
Funct	xxx	xxx	xxx	**100**	xxx	**011**	**000**	xxx	**111**	xxx	**101**	**110**	**001**	xxx	**010**

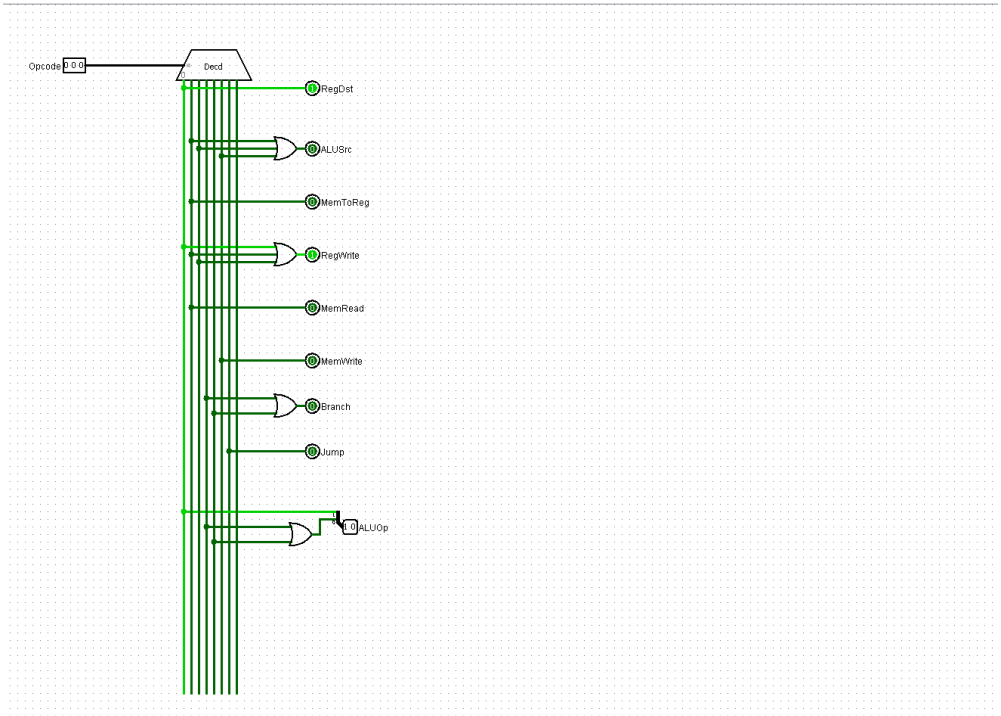
Register File



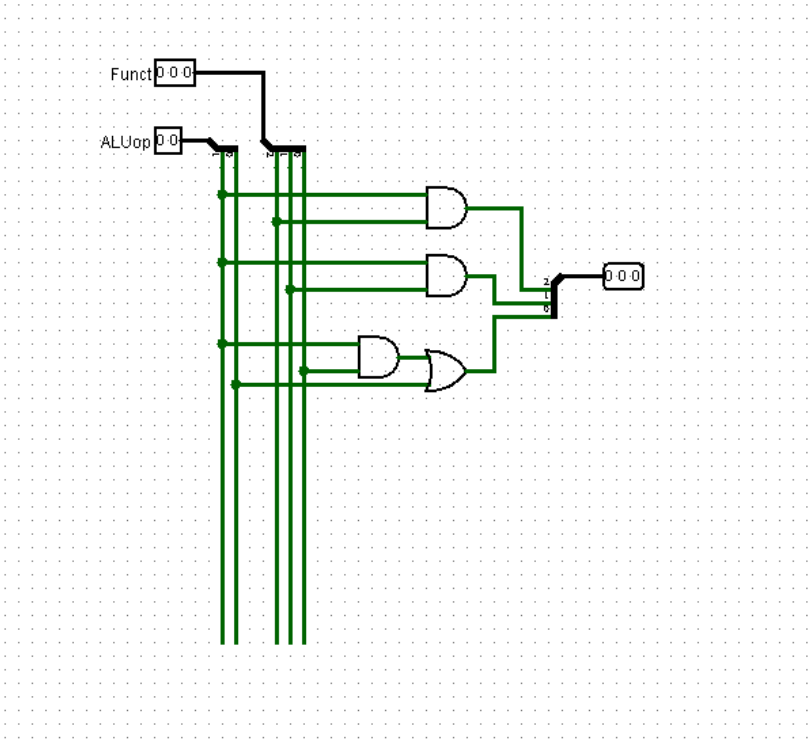
ALU



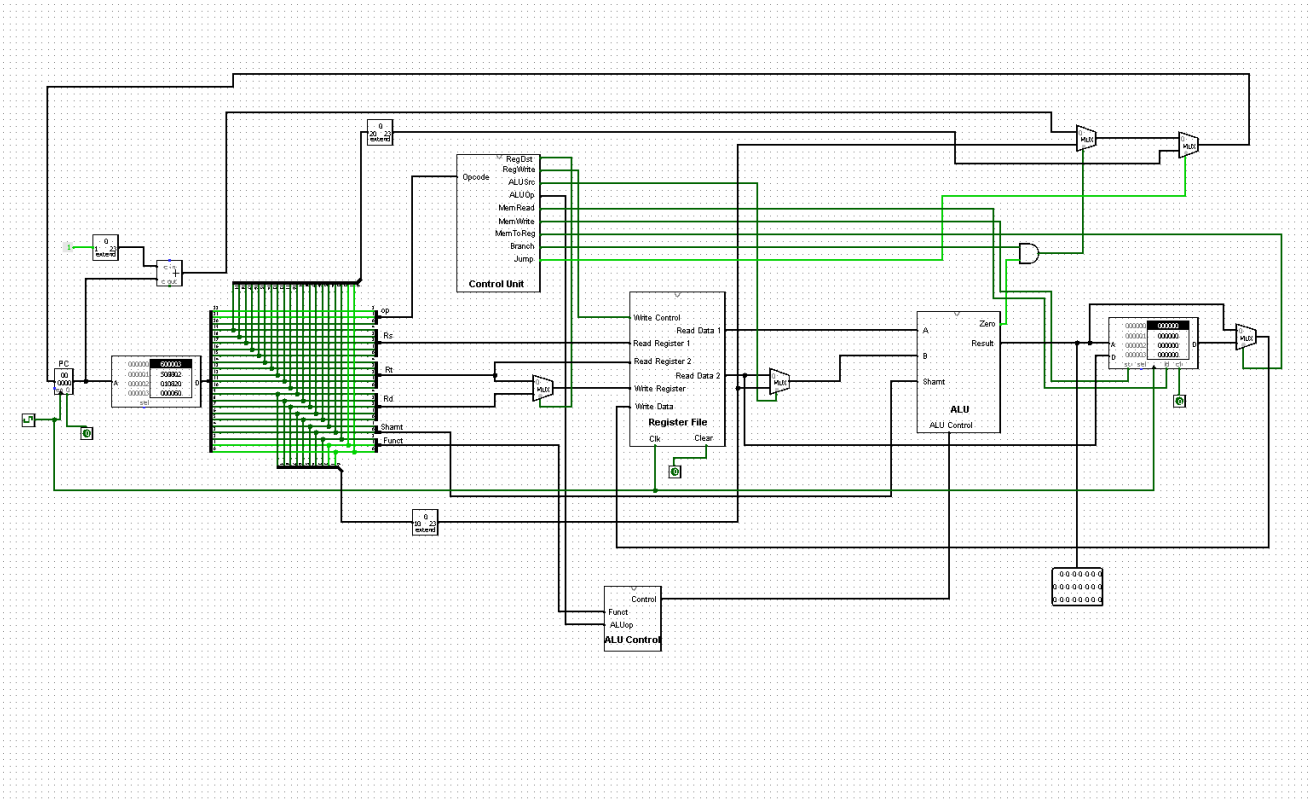
Control Unit



ALU Control Unit



Datapath


RegDst
RegWrite
ALUSrc
ALUOp
MemRead
MemWrite
MemToReg
Branch
Jump
Opcode
Control Unit
op
Rs
Rt
Rd
Shamt
Funct
Write Control
Read Register 1
Read Register 2
Write Register
Write Data
Read Data 1
Read Data 2
Clk
Clear
Register File
A
B
Shamt
Zero
Result
ALU
ALU Control
Control
Funct
ALUop
ALU Control
PC
MUX